

THE ICT UNIVERSITY

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGIES

SUMMER 2026 · FINAL EXAMINATION PROJECT REPORT

Lexical and Syntactic Analysis of Informal Urban Communication in Yaoundé

A regular-expression lexer and a table-driven LL(1) parser for code-mixed English, French, Pidgin and slang

Course	Details
Course title	Compiler Construction
Course code	CS4110 (credit value 6)
Instructor	Engr. Tanwi Nkiamboh
Department	ICT
Examination date	Tuesday, September 29, 2026

Group members

Name	Matricule	Role
NJUNG GLORIOUS SAMSIMBOM	ICTU20241398	Group leader
NDENGUE TAMOKOUE VAN BROWN	ICTU20241479	Member
BONGANG BERINYUY MEGHEN	ICTU20241385	Member

Source code: https://github.com/Samsi11/compiler_construction_yaounde

Table of Contents

1. Introduction	3
2. Methodology and data collection	3
2.1 How the data were collected	3
2.2 Data preparation	3
2.3 Coverage of the topics in the brief	4
2.4 Tools	4
3. Raw collected statements	4
3.1 Usable utterances	4
3.2 Entries excluded from analysis	6
4. Lexical analysis	6
4.1 Token types	6
4.2 Custom lexical specification (regular expressions)	7
4.3 Token tables	8
4.4 Token frequency and variation	9
5. Syntactic analysis	12
5.1 Designing the context-free grammar	12
5.2 Step 1: removing left recursion	13
5.3 Step 2: left factoring	13
5.4 Step 3: FIRST and FOLLOW sets	14
5.5 Step 4: the LL(1) parsing table	16
6. Implementation	18
6.1 Architecture	18
6.2 The table-driven LL(1) parsing algorithm	18
6.3 Worked example: F04 “U don chop?”	18
7. Testing and results	20
7.1 Automated tests	20
7.2 Overall results	20
7.3 Accepted utterances	21
7.4 Rejected utterances and why	23
8. Why Yaoundé communication is linguistically complex	25
9. Limitations and future work	26
10. Conclusion	26
References	26
Appendix A: Vocabulary	26
Appendix B: Running the project	27

Summary

This report describes a mini-language analyzer for informal speech in Yaoundé. A regular-expression **lexer** turns each phrase into typed, language-tagged tokens. A **context-free grammar** (CFG) is transformed (left-recursion removal, left factoring), its **FIRST/FOLLOW** sets are computed, and an **LL(1) parsing table** drives a **table-driven parser**. The system was tested on **50 real utterances** from a nine-person survey: **31 were accepted** and **19 were rejected**, each rejection for a documented linguistic reason.

1. Introduction

Yaoundé is a multilingual city. Everyday speech mixes English, French, Pidgin, local-language expressions and youth slang (“franc-anglais”), together with city-specific abbreviations such as brand and utility names. A single short sentence can draw on three or four of these at once, which makes such speech a hard test for tools built around one standard language.

The objective of this project, set out in the examination brief, is to design and implement a mini-language analyzer that performs lexical and syntactic analysis on real expressions from Yaoundé and detects patterns related to commuting, markets, electricity, weather, transport and everyday transactions. We answer it in five steps:

- **Collect** real phrases with the contributor’s own translation (Sections 2 and 3).
- **Analyze lexically**: define token types with regular expressions, tag each token with its language, and measure frequency and variation (Section 4).
- **Analyze syntactically**: build a CFG, remove left recursion, left-factor it, compute FIRST/FOLLOW and build the LL(1) table (Section 5).
- **Implement** a table-driven LL(1) parser with a desktop interface (Section 6).
- **Test** the grammar on the collected utterances and explain every acceptance and rejection (Section 7).

Section 8 discusses why Yaoundé communication is linguistically complex, and Section 9 lists limitations.

Throughout, a shaded **Comment** box under each figure or table explains what it shows and why it matters.

2. Methodology and data collection

2.1 How the data were collected

Data were gathered through an online survey form completed by **nine contributors** on 14 September 2026. The form was organised by topic (taxi and bendskin talk, money matters, rainy season, poor internet and network, market bargaining, electricity, phone credit and mobile money, food, fuel queues and an open topic). For each topic the contributor typed a phrase they had heard or used and, next to it, a plain-English translation.

Phrases were kept **exactly as typed**, including slang, code-mixing, spelling variants, missing punctuation and emoji. Nothing was corrected. Contributors are referred to by codes (P01 to P09) to protect their privacy.

Comment: how this relates to the brief

The brief asks for transcribed spoken statements. Our data are **written survey responses** describing phrases that contributors heard or used, so they show the same slang, mixing and mistakes but are not audio transcriptions. We state this openly as a limitation (Section 9).

Presentation clue: say that 50 usable utterances is well above the 10 to 15 required, and that the nine contributors give more variety than one group of three could.

2.2 Data preparation

- Each phrase became one row of `corpus.csv` with its topic, contributor code, translation and expected parser verdict.

- One form cell held two phrases on separate lines ("I drop aza?" and "Fund my account abeg"); it was split into two utterances (E07, E08).
- The phrase for "Massa Eneo" had no translation; the team supplied it from local usage (a sigh of annoyance when the electricity company Eneo cuts the power).
- **10 entries were excluded:** empty answers, entries with no translation, and one contributor whose topic keyword had been typed in the phrase column (columns apparently swapped). They are listed in Section 3.2.
- The result is **50 usable utterances** from seven of the nine contributors (P01 and P03 gave nothing usable). Utterances per contributor: P02 8, P04 5, P05 10, P06 5, P07 9, P08 4, P09 9.

2.3 Coverage of the topics in the brief

Topic in the brief	Status	Where it appears in our data
Taxi and commuting issues	Covered	Taxi / bendskin talk column (6 utterances)
Poor internet connectivity	Covered	Internet / network (6)
Limited electricity supply	Covered	Electricity (4)
Market bargaining	Covered	Market bargaining (6)
Rainy season struggles	Covered	Rainy season (7)
Fuel scarcity	Covered	Fuel queues (4)
Roadside business interactions	Partly	Through food and money-matters phrases
Bendskin communication	Partly	Shares one survey column with taxi talk
Security announcements or checkpoints	Not covered	No submissions received
Life at the ICT University	Not covered	No submissions received
(extra) Money, mobile money, food	Added	Money matters (8), mobile money (3), food (5), open (1)

Comment

Two suggested topics (security checkpoints and university life) received no submissions, so we do not claim coverage of them. The brief lists topics "such as", so it is not an exhaustive requirement.

2.4 Tools

The analyzer is written in **Python 3** using only the standard library (re, csv, tkinter, unittest). The lexical rules are read from an editable vocabulary file, and the whole project is kept in a Git repository so results can be reproduced with a few commands (Appendix B).

3. Raw collected statements

3.1 Usable utterances

The table lists every usable utterance **verbatim**, with the translation the contributor gave. Spelling, capitalisation, spacing and emoji are unchanged.

ID	Topic	By	Statement (as typed)	Contributor's translation
T01	Taxi / bendskin	P02	Fou le camp	Leave this place
T02	Taxi / bendskin	P04	Yu no serios	You're not serious
T03	Taxi / bendskin	P05	Enlève ta brouette la sur la route	Remove your trash car from the road
T04	Taxi / bendskin	P06	Lef me for that achombo house	Drop me off and that local restaurant
T05	Taxi / bendskin	P07	I get na 400	You are trying to bargain from the price the bike man gave you to 400
T06	Taxi / bendskin	P09	Na how	How are you
E01	Money matters	P02	Ne joue pas avec Mon argent salo	Don't play with my money fool

ID	Topic	By	Statement (as typed)	Contributor's translation
E02	Money matters	P04	That do small	That money is small
E03	Money matters	P05	Gère moi non nkap	Give back my money
E04	Money matters	P06	You get some twenty pongos for Dey ?	Do you have twenty thousand francs
E05	Money matters	P07	I send my aza make you fund me fapnkolo	Should I send my account details so you can send me 5000frs
E06	Money matters	P09	A fit see some small 100k 🤔🤔🤔	Could you please borrow me some money ,100k perhaps 🤔🤔
E07	Money matters	P08	I drop aza?	Should I send my account details?
E08	Money matters	P08	Fund my account abeg	Please can you send me some money
R01	Rainy season	P02	Na which bad luck this too nor?	What type of bad luck is this?
R02	Rainy season	P04	Rain do worry bad	Rain is disturbing
R03	Rainy season	P05	Massah yacob il fait froid	Places are cold
R04	Rainy season	P07	Enter motomoto	Enter mud
R05	Rainy season	P08	Motomoto	Mud
R06	Rainy season	P09	Do rain so eh	This rain like this eh
R07	Rainy season	P06	This rain so na weather four two and. A half	During this rainy season the rain and cold warrants you to cuddle up with a Partner
I01	Internet / network	P02	Mtn Va tous nous montrer au pays ci	Mtn will show us all types of things in this country
I02	Internet / network	P04	Network poor bad	The network is poor
I03	Internet / network	P05	Orange et le réseau nkangian	Orange and weak internet
I04	Internet / network	P06	MTN ns craze	The isp called MTN has very poor service
I05	Internet / network	P07	Yehum MTN	Disappointed at MTN
I06	Internet / network	P09	Make single pole go offline make network cam back	Let single people go offline so we can have good network 🤔
K01	Market bargaining	P02	Venez me tromper	Come and cheat me
K02	Market bargaining	P04	Levam for this price	Leave it at this price
K03	Market bargaining	P05	Majumbum depuis longtemps	I have money since
K04	Market bargaining	P07	Last market	What they sell around the closing time of the market
K05	Market bargaining	P08	Last price	Minimum price the seller can't go Below
K06	Market bargaining	P09	Ei over deer	It's too expensive
L01	Electricity	P02	Massa Eneo	Ugh, this Eneo (a sigh of annoyance when Eneo cuts the electricity)
L02	Electricity	P05	E.N.E.O	Eneo has done it again
L03	Electricity	P07	Ndon ya SOCADEL	Disappointed at SOCADEL
L04	Electricity	P09	Light no dey	There's no light
C01	Phone credit & Mobile Money	P05	Gère ton OM	Give me your orange money
C02	Phone credit & Mobile Money	P07	Abeg momo me	Please send me money through mobile money
C03	Phone credit & Mobile Money	P09	Abeg momo me 2k	Please send me 2k via momo
F01	Chop / food	P02	La nourriture si dja	This food is nice
F02	Chop / food	P05	Chop rice chop rice	Eat rice and don't touch my meat
F03	Chop / food	P06	I di find. Grind	I'm looking for food
F04	Chop / food	P07	U don chop?	Have you eaten?
F05	Chop / food	P09	You get chop for house	Do you have any food at home
U01	Fuel queues	P02	Fair vite nor	Do fast
U02	Fuel queues	P05	Le carburant est chère	Fuel is expensive
U03	Fuel queues	P07	Petrol don short	The available amount of fuel can not go round
U04	Fuel queues	P09	Petrol don deer	Petrol is now so expensive
O01	Open topic	P05	Le pater la	The president

Comment

This is the raw material for everything that follows: the lexical rules, the vocabulary and the grammar were all derived from these lines. Notice the mix inside single lines, such as French negation with a slang word (E01) or Pidgin verbs with English nouns (T04).

3.2 Entries excluded from analysis

ID	By	Phrase typed	Translation typed	Reason for exclusion
X01	P01	Taxi	(none)	Single word equal to the topic label; no translation given.
X02	P03	Taxi	Leave me at the junction	Topic label in the phrase column; the 'meaning' looks like the real utterance (columns appear swapped).
X03	P03	Money	Lent me some money	Same swapped-column pattern as X02.
X04	P03	Rain	The roads are destroyed by the rain	Same swapped-column pattern as X02.
X05	P03	Poor network	Network is poor	Same swapped-column pattern as X02.
X06	P03	Bargain	Reduce the price	Same swapped-column pattern as X02.
X07	P03	Electricity	Light left since in the morning	Same swapped-column pattern as X02.
X08	P03	Credit	(none)	Single word equal to the topic label; no translation given.
X09	P03	Food	Any new words	Topic label with a non-answer as the meaning.
X10	P09	No	(none)	No translation given.

4. Lexical analysis

4.1 Token types

A token is a word (or symbol) labelled with a **type** that says what grammatical role it can play. The lexer produces 18 token types. Each word also carries a **language tag** (EN English, FR French, PDG Pidgin, SLG local slang), which is how code-mixing is detected.

Token type	Meaning	Examples from the data
NOUN	Common and proper nouns, including slang nouns	nkap, house, achombo, réseau, Eneo
VERB	Main verbs (imperatives and finite forms)	chop, lef, momo, fund, gère
PRON	Personal pronouns	I, yu, u, moi, il
DET	Articles, possessives, demonstratives	le, la, my, that, tous
ADJ	Adjectives	small, deer, chère, nkangian, craze
ADV	Adverbs and intensifiers	how, vite, back, offline, over
AUX	Tense and aspect auxiliaries	don, di, fit, do, va
COP	Copulas (“is”)	na, est, si
NEG	Negation words	no, pas, non
NEG_PRE	First half of French ne ... pas	ne
PREP	Prepositions	for, avec, sur, au, depuis
CONJ	Conjunctions	make, et, and
NUM	Number words and numeric literals	twenty, four, 400, 2k, 100k
SLANG	Exclamations and discourse slang	abeg, massa, ndon ya, ei, salo
PART	Sentence-final particles	nor, eh
QMARK	The question mark	?
WORD	A word that is not in the vocabulary	yacob, ns, half
INVALID	Any other character (emoji, stray full stop)	🤔, .

Comment: how the brief's categories map to our design

Nouns and verbs are the NOUN and VERB types. **Slang words** (exclamations such as abeg, massa and yehum) are the SLANG type, plus slang nouns such as nkap that carry the tag SLG. **Code-mixed expressions** are found by the language tag on every token: an utterance with tokens from two or more languages is code-mixed.

Presentation clue: 37 of 50 utterances (74.0%) are code-mixed.

Examples of code-mixed utterances, with the language of each word in brackets:

ID	Words with language tag	Languages
T04	Lef(PDG) me(EN) for(EN) that(EN) achombo(SLG) house(EN)	English + Pidgin + Slang
E01	Ne(FR) joue(FR) pas(FR) avec(FR) Mon(FR) argent(FR) salo(SLG)	French + Slang
I03	Orange(EN) et(FR) le(FR) réseau(FR) nkangian(SLG)	English + French + Slang
E05	I(EN) send(EN) my(EN) aza(SLG) make(PDG) you(EN) fund(EN) me(EN) fapnkolo(SLG)	English + Pidgin + Slang
I06	Make(PDG) single(EN) pole(PDG) go(EN) offline(EN) make(PDG) network(EN) cam(PDG) back(EN)	English + Pidgin
U01	Fair(FR) vite(FR) nor(PDG)	French + Pidgin
K06	Ei(PDG) over(EN) deer(PDG)	English + Pidgin
F01	La(FR) nourriture(FR) si(UNK) dja(UNK)	French

4.2 Custom lexical specification (regular expressions)

The lexical specification works like a LEX/FLEX rule file. Each token type has **its own regular expression**, built from the words listed for that type in `vocabulary.csv` (134 entries). Four generic rules follow for numbers, the question mark, unknown words and any other character. All rules are joined into one master pattern and tried in order; **the first rule that matches wins**.

#	Token type	Regular expression (L = [A-Za-zÀ-ÿ])	Words
1	VERB	(?<!L)(?i:tromper montrer enlève venez enter worry ...)(?!L)	24
2	NOUN	(?<!L)(?i:nouriture carburant brouette majumbum ...)(?!L)	36
3	PRON	(?<!L)(?i:nous you moi me yu i i u a)(?!L)	9
4	DET	(?<!L)(?i:which tous that this some ton mon le la ta my)(?!L)	11
5	ADJ	(?<!L)(?i:nkangian single serios small short froid ...)(?!L)	13
6	ADV	(?<!L)(?i:longtemps offline back over vite how too so ci)(?!L)	9
7	AUX	(?<!L)(?i:don fit do di va)(?!L)	5
8	COP	(?<!L)(?i:est na si)(?!L)	3
9	NEG	(?<!L)(?i:pas non no)(?!L)	3
10	NEG_PRE	(?<!L)(?i:ne)(?!L)	1
11	PREP	(?<!L)(?i:depuis avec for sur au)(?!L)	5
12	CONJ	(?<!L)(?i:make and et)(?!L)	3
13	NUM	(?<!L)(?i:twenty four two)(?!L)	3
14	SLANG	(?<!L)(?i:ndon\s+ya massah massa yehum salo abeg ei)(?!L)	7
15	PART	(?<!L)(?i:nor eh)(?!L)	2
16	NUM	\d+(?:[.,]\d+)?(?i:frs fcfa f k)?(?![A-Za-zÀ-ÿ0-9])	–
17	QMARK	\?	–
18	WORD	L+(?:'L+)?	–
19	INVALID	\S	–

- (?<!L) and (?!L) are **word-boundary checks**: a short word such as a can never match inside sala.
- (?i:...) makes matching **case-insensitive**, so Le, le and LE are the same word.
- A two-word entry such as ndon ya becomes ndon\s+ya, so it is **one token** whatever the spacing.
- The last rule \S catches any leftover character and labels it INVALID, so **nothing is silently dropped**.

Comment

Adding a word to `vocabulary.csv` regenerates the regular expression for its type automatically. The rules are therefore a real lexical specification (regular expressions per token type) that grows with the data rather than a fixed list in code.

Presentation clue: explain that the lexer is a deterministic scanner, exactly what LEX would generate, only written in Python.

Vocabulary size by token type and language:

Type	EN	FR	PDG	SLG	UNK	Total
NOUN	16	10	2	8	0	36
VERB	9	9	6	0	0	24
PRON	3	3	3	0	0	9
DET	5	6	0	0	0	11
ADJ	6	2	3	1	1	13
ADV	6	3	0	0	0	9
AUX	0	1	4	0	0	5
COP	0	1	1	0	1	3
NEG	0	2	1	0	0	3
NEG_PRE	0	1	0	0	0	1
PREP	1	4	0	0	0	5
CONJ	1	1	1	0	0	3
NUM	3	0	0	0	0	3
SLANG	0	0	2	5	0	7
PART	0	0	2	0	0	2

4.3 Token tables

Every utterance was tokenized. Each token is shown as **word** followed by its TYPE and language (·FR, ·PDG, ·SLG, ·EN). Tokens without a language are numbers, punctuation and unknown words.

ID	Tokens
T01	Fou VERB-FR le DET-FR camp NOUN-FR
T02	Yu PRON-PDG no NEG-PDG serios ADJ-PDG
T03	Enlève VERB-FR ta DET-FR brouette NOUN-FR la DET-FR sur PREP-FR la DET-FR route NOUN-FR
T04	Lef VERB-PDG me PRON-EN for PREP-EN that DET-EN achombo NOUN-SLG house NOUN-EN
T05	I PRON-EN get VERB-EN na COP-PDG 400 NUM
T06	Na COP-PDG how ADV-EN
E01	Ne NEG-PR-EN joue VERB-FR pas NEG-FR avec PREP-FR Mon DET-FR argent NOUN-FR salo SLANG-SLG
E02	That DET-EN do AUX-PDG small ADJ-EN
E03	Gère VERB-FR moi PRON-FR non NEG-FR nkap NOUN-SLG
E04	You PRON-EN get VERB-EN some DET-EN twenty NUM-EN pongos NOUN-SLG for PREP-EN Dey VERB-PDG ? QMARK
E05	I PRON-EN send VERB-EN my DET-EN aza NOUN-SLG make CONJ-PDG you PRON-EN fund VERB-EN me PRON-EN fapnkolo NOUN-SLG
E06	A PRON-PDG fit AUX-PDG see VERB-EN some DET-EN small ADJ-EN 100k NUM  INVALID  INVALID  INVALID
E07	I PRON-EN drop VERB-EN aza NOUN-SLG ? QMARK
E08	Fund VERB-EN my DET-EN account NOUN-EN abeg SLANG-PDG
R01	Na COP-PDG which DET-EN bad ADJ-EN luck NOUN-EN this DET-EN too ADV-EN nor PART-PDG ? QMARK
R02	Rain NOUN-EN do AUX-PDG worry VERB-EN bad ADJ-EN
R03	Massah SLANG-SLG yacob WORD il PRON-FR fait VERB-FR froid ADJ-FR
R04	Enter VERB-EN motomoto NOUN-SLG
R05	Motomoto NOUN-SLG
R06	Do AUX-PDG rain NOUN-EN so ADV-EN eh PART-PDG
R07	This DET-EN rain NOUN-EN so ADV-EN na COP-PDG weather NOUN-EN four NUM-EN two NUM-EN and CONJ-EN .INVALID A PRON-PDG half WORD

ID	Tokens
I01	Mtn NOUN-EN Va AUX-FR tous DET-FR nous PRON-FR montrer VERB-FR au PREP-FR pays NOUN-FR ci ADV-FR
I02	Network NOUN-EN poor ADJ-EN bad ADJ-EN
I03	Orange NOUN-EN et CONJ-FR le DET-FR réseau NOUN-FR nkangian ADJ-SLG
I04	MTN NOUN-EN ns WORD craze ADJ-PDG
I05	Yehum SLANG-SLG MTN NOUN-EN
I06	Make CONJ-PDG single ADJ-EN pole NOUN-PDG go VERB-EN offline ADV-EN make CONJ-PDG network NOUN-EN cam VERB-PDG back ADV-EN
K01	Venez VERB-FR me PRON-EN tromper VERB-FR
K02	Levam VERB-PDG for PREP-EN this DET-EN price NOUN-EN
K03	Majumbum NOUN-SLG depuis PREP-FR longtemps ADV-FR
K04	Last ADJ-EN market NOUN-EN
K05	Last ADJ-EN price NOUN-EN
K06	Ei SLANG-PDG over ADV-EN deer ADJ-PDG
L01	Massa SLANG-SLG Eneo NOUN-EN
L02	E.N.E.O NOUN-EN
L03	Ndon ya SLANG-SLG SOCADEL NOUN-EN
L04	Light NOUN-EN no NEG-PDG dey VERB-PDG
C01	Gère VERB-FR ton DET-FR OM NOUN-FR
C02	Abeg SLANG-PDG momo VERB-PDG me PRON-EN
C03	Abeg SLANG-PDG momo VERB-PDG me PRON-EN 2k NUM
F01	La DET-FR nourriture NOUN-FR si COP dja ADJ
F02	Chop VERB-PDG rice NOUN-EN chop VERB-PDG rice NOUN-EN
F03	I PRON-EN di AUX-PDG find VERB-EN . INVALID Grind NOUN-PDG
F04	U PRON-PDG don AUX-PDG chop VERB-PDG ? QMARK
F05	You PRON-EN get VERB-EN chop VERB-PDG for PREP-EN house NOUN-EN
U01	Fair VERB-FR vite ADV-FR nor PART-PDG
U02	Le DET-FR carburant NOUN-FR est COP-FR chère ADJ-FR
U03	Petrol NOUN-EN don AUX-PDG short ADJ-EN
U04	Petrol NOUN-EN don AUX-PDG deer ADJ-PDG
O01	Le DET-FR pater NOUN-FR la DET-FR

Comment

Read one row as the lexer output that the parser receives. Rejected utterances still tokenize: for example R03 contains yacob as **WORD** (unknown), and E06 ends with three **INVALID** emoji tokens.

4.4 Token frequency and variation

The statistics below are computed by `analysis.py` over the 50 usable utterances.

Measure	Value	Meaning
Utterances	50	Usable phrases analysed
Tokens	211 (average 4.22 per utterance)	Utterances are short, fragmentary speech
Type-token ratio	0.688	139 distinct words / 202 word tokens; higher means more varied
Words used only once	101 of 139 (72.7%)	Most vocabulary is rare, so a fixed word list keeps growing
Code-mixing ratio	56.7%	Share of language-tagged tokens that are not English
Code-mixed utterances	37 of 50 (74.0%)	Utterances with two or more languages
Out-of-vocabulary words	3	yacob, half, ns

Interpretation

- **Function words dominate the top ranks** (me 5; le, la, for, i, na, chop 4 each) while 72.7% of distinct words appear once. Content vocabulary is diverse, which is why the lexicon has to be data-driven.

- **English is a minority:** only 84 of 194 tagged tokens (43.3%) are English; French is 25.8%, Pidgin 23.2% and local slang 7.7%.
- **One meaning, many forms:** money is written argent (French), nkap, pongos, fapnkolo and majumbum (slang); transferring money is gère, fund, send, drop and momo. A parser that relies on one spelling would miss most of them.
- **Money is the most mixed topic:** all 8 money utterances mix two or more languages.

Token types

Type	Count	Share
NOUN	46	21.8%
VERB	33	15.6%
DET	22	10.4%
PRON	19	9.0%
ADJ	18	8.5%
ADV	10	4.7%
SLANG	9	4.3%
AUX	9	4.3%
PREP	8	3.8%
COP	6	2.8%
NUM	6	2.8%
CONJ	5	2.4%
INVALID	5	2.4%
NEG	4	1.9%
QMARK	4	1.9%
PART	3	1.4%
WORD	3	1.4%
NEG_PRE	1	0.5%

Language of tagged tokens

Language	Count	Share
English (EN)	84	43.3%
French (FR)	50	25.8%
Pidgin (PDG)	45	23.2%
Slang (SLG)	15	7.7%

Top 15 words

Word	Count
me	5
le	4
la	4
for	4
i	4
na	4
chop	4
get	3
do	3
you	3
make	3
abeg	3
bad	3
this	3
rain	3

Utterances, tokens and code-mixed utterances by topic:

Topic	Utterances	Tokens	Code-mixed
Electricity	4	8	3
Chop / food	5	22	3
Fuel queues	4	13	3
Internet / network	6	30	5
Market bargaining	6	17	4
Phone credit & Mobile Money	3	10	2
Money matters	8	48	8
Open topic	1	3	0
Rainy season	7	35	6
Taxi / bendskin	6	25	3

Concept variation: the same meaning expressed with different surface forms (language in brackets, × = times seen).

Concept	Surface forms found
ACCOUNT	aza [SLG] ×2, account [EN] ×1
EXPENSIVE	deer [PDG] ×2, chère [FR] ×1
FOOD	nouriture [FR] ×1, grind [PDG] ×1
FUEL	petrol [EN] ×2, carburant [FR] ×1
LEAVE	fou [FR] ×1, lef [PDG] ×1, levam [PDG] ×1
MONEY	argent [FR] ×1, nkap [SLG] ×1, pongos [SLG] ×1, fapnkolo [SLG] ×1, majumbum [SLG] ×1
NETWORK	network [EN] ×2, réseau [FR] ×1
TRANSFER	gère [FR] ×2, fund [EN] ×2, momo [PDG] ×2, send [EN] ×1, drop [EN] ×1
WEAK_SERVICE	poor [EN] ×1, nkangian [SLG] ×1, craze [PDG] ×1

```
PS C:\Users\Glorious Samsibom\compiler_construction_yaounde> py main.py --grammar --no-interactive
PS C:\Users\Glorious Samsibom\compiler_construction_yaounde> py main.py --stats --no-interactive
=== Yaounde Urban Language Analyzer ===

=== Token frequency & variation analysis ===
Utterances analyzed : 50
Total tokens       : 211 (average 4.2 per utterance)
Type-token ratio   : 0.688 (distinct words / total words; higher = more lexically varied)
Code-mixing ratio  : 56.7% of language-tagged tokens are French/Pidgin/slang rather than English
Code-mixed utterances: 37 of 50 contain words from two or more languages

Token types (POS):
NOUN   46
VERB   33
DET    22
PRON   19
ADJ    18
ADV    10
SLANG   9
AUX     9
PREP    8
COP     6
NUM     6
CONJ    5
INVALID 5
NEG     4
QMARK  4
PART    3
WORD    3
NEG_PRE 1

Language of tagged tokens (EN English, FR French, PDG Pidgin, SLG local slang):
EN      84
FR      50
PDG     45
SLG     15
```

Figure 1: Output of `py main.py --stats` (summary, token types, languages).

```
PS C:\Users\Glorious Samsibom\compiler_construction_yaounde> py main.py --stats --no-interactive
NEG_PRE 1

Language of tagged tokens (EN English, FR French, PDG Pidgin, SLG local slang):
EN      84
FR      50
PDG     45
SLG     15

Top 15 most frequent words:
me      5
le      4
la      4
for     4
i       4
na      4
chop    4
get     3
do      3
you     3
make    3
abog    3
bad     3
this    3
rain    3

By topic (utterances / tokens / code-mixed utterances):
electricity 4 / 8 / 3
food        5 / 22 / 3
fuel        4 / 13 / 3
internet    6 / 30 / 5
market      6 / 17 / 4
mobile_money 3 / 10 / 2
money       8 / 48 / 8
open        1 / 3 / 0
rain        7 / 35 / 6
taxi        6 / 25 / 3

Concept variation (same meaning, different surface forms):
```

Figure 2: Same command: top words and counts by topic.

```
PS C:\Users\Glorious Samsibom\compiler_construction_yaounde> py main.py --stats --no-interactive
money      8 / 48 / 8
open       1 / 3 / 0
rain       7 / 35 / 6
taxi       6 / 25 / 3

Concept variation (same meaning, different surface forms):
ACCOUNT    aza [SLG] x2, account [EN] x1
EXPENSIVE  deer [PDG] x2, chère [FR] x1
FOOD       nouriture [FR] x1, grind [PDG] x1
FUEL       petrol [EN] x2, carburant [FR] x1
LEAVE      fou [FR] x1, lef [PDG] x1, levam [PDG] x1
MONEY      argent [FR] x1, nkap [SLG] x1, pongos [SLG] x1, fapnkolo [SLG] x1, majumbum [SLG] x1
NETWORK    network [EN] x2, réseau [FR] x1
TRANSFER   gère [FR] x2, fund [EN] x2, momo [PDG] x2, send [EN] x1, drop [EN] x1
WEAK_SERVICE poor [EN] x1, nkangian [SLG] x1, craze [PDG] x1

Out-of-vocabulary words (tagged WORD):
yacob     1
half      1
ns        1

=== Parsing 50 collected utterances ===

T01 [taxi, P02]: 'Fou le camp'
Tokens: [('Fou', 'VERB', 'FR'), ('le', 'DET', 'FR'), ('camp', 'NOUN', 'FR')]
Result: Syntax Valid (Accepted by CFG)
Contributor's translation: Leave this place

T02 [taxi, P04]: 'Yu no serios'
Tokens: [('Yu', 'PRON', 'PDG'), ('no', 'NEG', 'PDG'), ('serios', 'ADJ', 'PDG')]
Result: Syntax Valid (Accepted by CFG)
Contributor's translation: You're not serious

T03 [taxi, P05]: 'Enlève ta brouette la sur la route'
Tokens: [('Enlève', 'VERB', 'FR'), ('ta', 'DET', 'FR'), ('brouette', 'NOUN', 'FR'), ('la', 'DET', 'FR'), ('route', 'NOUN', 'FR')]
Result: Syntax Error near token 'la' (unexpected DET; expected one of: $, ADJ, AUX, CONJ, COP, NEG, NE RB) [Rejected]
```

Figure 3: Same command: concept variation and out-of-vocabulary words.

Comment

These screenshots are the program's own output, so every number in the tables above can be reproduced with `py main.py --stats --no-interactive`.

Presentation clue: the out-of-vocabulary list is short (yacob, ns, half) only because the vocabulary was built from these same utterances. On new speech it would be longer, which we say honestly in the limitations.

5. Syntactic analysis

5.1 Designing the context-free grammar

A context-free grammar (CFG) is a set of rewrite rules that describes which sequences of token types form a valid sentence. We looked at the accepted shapes in the data and wrote rules for them:

- **Imperatives:** Fou le camp; Enter motomoto; Gère ton OM (verb + object).
- **Subject + predicate:** Yu no serio; Petrol don short; Le carburant est chère (copula, aspect marker don, negation no).
- **French negation ne ... pas:** Ne joue pas avec Mon argent salo.
- **Double objects and clause chains:** I send my aza make you fund me fapnkolo.
- **Fragments, exclamations and particles:** Last price; Massa Eneo; Fair vite nor.

Symbol	Stands for
S	a whole utterance
LEAD	optional leading exclamation
UTT	one or more clauses joined by a conjunction
CLAUSE	one clause (fragment, imperative or subject + predicate)
PRED	what is said about the subject
AUXC	what follows an aspect auxiliary
NEGC	what follows a negation word
COMP	complement of a copula
ADJP	one or more adjectives
VP	verb phrase (with French ne...pas if present)
VP1	first thing after the verb
VPOBJ	second object or trailing PPs
VP2	any number of prepositional phrases
PP	preposition + noun phrase
NP	noun phrase
NPN	non-pronoun noun phrase
NBAR	noun core: adjectives, numerals and noun compounds
TAIL	final particle / exclamation and question mark
TAILW	the final particle or exclamation
QM	optional question mark

The CFG as designed (ϵ means “nothing”; upper-case words that never appear on the left are token types):

```

S -> LEAD UTT TAIL
LEAD -> SLANG |  $\epsilon$ 
UTT -> UTT CONJ CLAUSE | CONJ CLAUSE | CLAUSE
CLAUSE -> NP PRED | NP | VP | COP COMP | ADV ADJP
PRED -> VP | COP COMP | AUX AUXC | NEG NEGC | ADJP
AUXC -> ADJP | VP
NEGC -> ADJP | VP
COMP -> ADJP | ADV | ADV ADJP
ADJP -> ADJP ADJ | ADJ
VP -> NEG_PRE VERB NEG VP1 | VERB VP1
VP1 -> PRON VPOBJ | NPN VP2 | PP VP2 | ADV VP2 |  $\epsilon$ 

```

```

VPOBJ -> NP VP2 | VP2
VP2 -> PP VP2 | ε
PP -> PREP NP
NP -> PRON | NPN
NPN -> DET NBAR | NBAR
NBAR -> NBAR NOUN | NOUN | ADJ NOUN | ADJ NUM | NUM
TAIL -> TAILW QM
TAILW -> PART | SLANG | ε
QM -> QMARK | ε

```

Comment

The grammar is deliberately a **fragment**: it describes the shapes seen most often, not the whole language. Utterances outside these shapes are rejected on purpose, which is discussed in Sections 7 and 8.

Presentation clue: S is the start symbol; LEAD and TAIL wrap the optional exclamation at the start and the particle or question mark at the end.

5.2 Step 1: removing left recursion

A rule such as $A \rightarrow A x \mid y$ calls itself first (left recursion). A top-down parser would expand A forever without consuming a token. The standard fix rewrites $A \rightarrow A x \mid y$ as $A \rightarrow y A'$ and $A' \rightarrow x A' \mid \epsilon$. Three rules were left-recursive:

Rule	Before	After
UTT	UTT -> UTT CONJ CLAUSE CONJ CLAUSE CLAUSE	UTT -> CONJ CLAUSE UTT' CLAUSE UTT' UTT' -> CONJ CLAUSE UTT' ε
ADJP	ADJP -> ADJP ADJ ADJ	ADJP -> ADJ ADJP' ADJP' -> ADJ ADJP' ε
NBAR	NBAR -> NBAR NOUN NOUN ADJ NOUN ADJ NUM NUM	NBAR -> NOUN NBAR' ADJ NOUN NBAR' ADJ NUM NBAR' NUM NBAR' NBAR' -> NOUN NBAR' ε

```

PS C:\Users\Glorious Samsibom\compiler_construction_yaounde> py main.py --grammar --no-interactive
2. After left-recursion elimination (UTT, ADJP, NBAR were left-recursive):
S -> LEAD UTT TAIL
LEAD -> SLANG | ε
UTT -> CONJ CLAUSE UTT' | CLAUSE UTT'
CLAUSE -> NP PRED | NP | VP | COP COMP | ADV ADJP
PRED -> VP | COP COMP | AUX AUXC | NEG NEGC | ADJP
AUXC -> ADJP | VP
NEGC -> ADJP | VP
COMP -> ADJP | ADV | ADV ADJP
ADJP -> ADJ ADJP'
ADJP' -> ADJ ADJP'
VP -> NEG_PRE VERB NEG VP1 | VERB VP1
VP1 -> PRON VPOBJ | NPN VP2 | PP VP2 | ADV VP2 | ε
VPOBJ -> NP VP2 | VP2
VP2 -> PP VP2 | ε
PP -> PREP NP
NP -> PRON | NPN
NPN -> DET NBAR | NBAR
NBAR -> NOUN NBAR' | ADJ NOUN NBAR' | ADJ NUM NBAR' | NUM NBAR'
TAIL -> TAILW QM
TAILW -> PART | SLANG | ε
QM -> QMARK | ε
UTT' -> CONJ CLAUSE UTT' | ε

```

Figure 4: Program output for step 2 ('py main.py --grammar'): the grammar after left-recursion elimination.

Comment

UTT, ADJP and NBAR are the places where the data repeat a unit: several clauses, several adjectives, several nouns (achombo house). The helper symbols UTT', ADJP' and NBAR' express the repetition without left recursion. All other rules had none and were left unchanged by the algorithm.

5.3 Step 2: left factoring

When two rules for the same symbol start identically, one token of lookahead cannot decide which to use. Left factoring pulls out the shared start: $A \rightarrow x y \mid x z$ becomes $A \rightarrow x A'$ and $A' \rightarrow y \mid z$. Three symbols needed it:

Rule	Before (after step 1)	After
CLAUSE	CLAUSE -> NP PRED NP VP COP COMP ADV ADJP	CLAUSE -> VP COP COMP ADV ADJP NP CLAUSE' CLAUSE' -> PRED ε

Rule	Before (after step 1)	After
COMP	COMP -> ADJP ADV ADV ADJP	COMP -> ADJP ADV COMP' COMP' -> ε ADJP
NBAR	NBAR -> NOUN NBAR' ADJ NOUN NBAR' ADJ NUM NBAR' NUM NBAR'	NBAR -> NOUN NBAR' NUM NBAR' ADJ NBAR'' NBAR'' -> NOUN NBAR' NUM NBAR'

```

PS C:\Users\Glorious Samsimbom\compiler_construction_yaounde> py main.py --grammar --no-interactive

3. After Left-factoring (CLAUSE, COMP, NBAR shared prefixes):
S -> LEAD UTT TAIL
LEAD -> SLANG | ε
UTT -> CONJ CLAUSE UTT' | CLAUSE UTT'
CLAUSE -> VP | COP COMP | ADV ADJP | NP CLAUSE'
PRED -> VP | COP COMP | AUX AUXC | NEG NEGC | ADJP
AUXC -> ADJP | VP
NEGC -> ADJP | VP
COMP -> ADJP | ADV COMP'
ADJP -> ADJ ADJP'
VP -> NEG PRE VERB NEG VP1 | VERB VP1
VP1 -> PRON VPOBJ | NPN VP2 | PP VP2 | ADV VP2 | ε
VPOBJ -> NP VP2 | VP2
VP2 -> PP VP2 | ε
PP -> PREP NP
NP -> PRON | NPN
NPN -> DET NBAR | NBAR
NBAR -> NOUN NBAR' | NUM NBAR' | ADJ NBAR''
TAIL -> TAILW QM
TAILW -> PART | SLANG | ε
QM -> QMARK | ε
UTT' -> CONJ CLAUSE UTT' | ε
ADJP' -> ADJ ADJP' | ε
NBAR' -> NOUN NBAR' | ε
CLAUSE' -> PRED | ε
COMP' -> ε | ADJP
NBAR'' -> NOUN NBAR' | NUM NBAR'

```

Figure 5: Program output for step 3: the final LL(1)-ready grammar after left factoring.

Comment

CLAUSE is the clearest case: a subject noun phrase may or may not be followed by a predicate (NP PRED versus just NP). After factoring the parser reads the NP first and decides afterwards through CLAUSE'. This is the grammar the parser actually uses.

5.4 Step 3: FIRST and FOLLOW sets

FIRST(X) is the set of token types that can begin a string derived from X (ε is included if X can derive nothing).

FOLLOW(X) is the set of token types that can appear immediately after X. Both are computed by repeating the standard rules until nothing changes.

Symbol	FIRST	FOLLOW
S	ADJ, ADV, CONJ, COP, DET, NEG_PRE, NOUN, NUM, PRON, SLANG, VERB	\$
LEAD	SLANG, ε	ADJ, ADV, CONJ, COP, DET, NEG_PRE, NOUN, NUM, PRON, VERB
UTT	ADJ, ADV, CONJ, COP, DET, NEG_PRE, NOUN, NUM, PRON, VERB	\$, PART, QMARK, SLANG
CLAUSE	ADJ, ADV, COP, DET, NEG_PRE, NOUN, NUM, PRON, VERB	\$, CONJ, PART, QMARK, SLANG
PRED	ADJ, AUX, COP, NEG, NEG_PRE, VERB	\$, CONJ, PART, QMARK, SLANG
AUXC	ADJ, NEG_PRE, VERB	\$, CONJ, PART, QMARK, SLANG
NEGC	ADJ, NEG_PRE, VERB	\$, CONJ, PART, QMARK, SLANG
COMP	ADJ, ADV	\$, CONJ, PART, QMARK, SLANG
ADJP	ADJ	\$, CONJ, PART, QMARK, SLANG
VP	NEG_PRE, VERB	\$, CONJ, PART, QMARK, SLANG
VP1	ADJ, ADV, DET, NOUN, NUM, PREP, PRON, ε	\$, CONJ, PART, QMARK, SLANG
VPOBJ	ADJ, DET, NOUN, NUM, PREP, PRON, ε	\$, CONJ, PART, QMARK, SLANG
VP2	PREP, ε	\$, CONJ, PART, QMARK, SLANG
PP	PREP	\$, CONJ, PART, PREP, QMARK, SLANG
NP	ADJ, DET, NOUN, NUM, PRON	\$, ADJ, AUX, CONJ, COP, NEG, NEG_PRE, PART, PREP, QMARK, SLANG, VERB
NPN	ADJ, DET, NOUN, NUM	\$, ADJ, AUX, CONJ, COP, NEG, NEG_PRE, PART, PREP, QMARK, SLANG, VERB
NBAR	ADJ, NOUN, NUM	\$, ADJ, AUX, CONJ, COP, NEG, NEG_PRE, PART, PREP, QMARK, SLANG, VERB
TAIL	PART, QMARK, SLANG, ε	\$

Symbol	FIRST	FOLLOW
TAILW	PART, SLANG, ϵ	\$, QMARK
QM	QMARK, ϵ	\$
UTT'	CONJ, ϵ	\$, PART, QMARK, SLANG
ADJP'	ADJ, ϵ	\$, CONJ, PART, QMARK, SLANG
NBAR'	NOUN, ϵ	\$, ADJ, AUX, CONJ, COP, NEG, NEG_PRE, PART, PREP, QMARK, SLANG, VERB
CLAUSE'	ADJ, AUX, COP, NEG, NEG_PRE, VERB, ϵ	\$, CONJ, PART, QMARK, SLANG
COMP'	ADJ, ϵ	\$, CONJ, PART, QMARK, SLANG
NBAR''	NOUN, NUM	\$, ADJ, AUX, CONJ, COP, NEG, NEG_PRE, PART, PREP, QMARK, SLANG, VERB

```

PS C:\Users\Glorious Samsibom\compiler_construction_yaounde> py main.py --grammar --no-interactive
4. FIRST sets:
S: { ADJ, ADV, COND, COP, DET, NEG_PRE, NOUN, NUM, PRON, SLANG, VERB }
LEAD: { SLANG,  $\epsilon$  }
UTT: { ADJ, ADV, COND, COP, DET, NEG_PRE, NOUN, NUM, PRON, VERB }
CLAUSE: { ADJ, ADV, COP, DET, NEG_PRE, NOUN, NUM, PRON, VERB }
PREP: { ADJ, AUX, COP, NEG, NEG_PRE, VERB }
AUXC: { ADJ, NEG_PRE, VERB }
NEGC: { ADJ, NEG_PRE, VERB }
COMP: { ADJ, ADV }
ADJP: { ADJ }
VP: { NEG_PRE, VERB }
VP1: { ADJ, ADV, DET, NOUN, NUM, PREP, PRON,  $\epsilon$  }
VPOB3: { ADJ, DET, NOUN, NUM, PREP, PRON,  $\epsilon$  }
VP2: { PREP,  $\epsilon$  }
PP: { PREP }
NP: { ADJ, DET, NOUN, NUM, PRON }
NPN: { ADJ, DET, NOUN, NUM }
NBAR: { ADJ, NOUN, NUM }
TAIL: { PART, QMARK, SLANG,  $\epsilon$  }
TAILW: { PART, SLANG,  $\epsilon$  }
QM: { QMARK,  $\epsilon$  }
UTT': { CONJ,  $\epsilon$  }
ADJP': { ADJ,  $\epsilon$  }
NBAR': { NOUN,  $\epsilon$  }
CLAUSE': { ADJ, AUX, COP, NEG, NEG_PRE, VERB,  $\epsilon$  }
COMP': { ADJ,  $\epsilon$  }
NBAR'': { NOUN, NUM }

```

Figure 6: FIRST sets as printed by the program.

```

PS C:\Users\Glorious Samsibom\compiler_construction_yaounde> py main.py --grammar --no-interactive
5. FOLLOW sets:
S: { $ }
LEAD: { ADJ, ADV, COND, COP, DET, NEG_PRE, NOUN, NUM, PRON, VERB }
UTT: { $, PART, QMARK, SLANG }
CLAUSE: { $, COND, PART, QMARK, SLANG }
PREP: { $, COND, PART, QMARK, SLANG }
AUXC: { $, COND, PART, QMARK, SLANG }
NEGC: { $, COND, PART, QMARK, SLANG }
COMP: { $, COND, PART, QMARK, SLANG }
ADJP: { $, COND, PART, QMARK, SLANG }
VP: { $, COND, PART, QMARK, SLANG }
VP1: { $, COND, PART, QMARK, SLANG }
VPOB3: { $, COND, PART, QMARK, SLANG }
VP2: { $, COND, PART, QMARK, SLANG }
PP: { $, COND, PART, PREP, QMARK, SLANG }
NP: { $, ADJ, AUX, COND, COP, NEG, NEG_PRE, PART, PREP, QMARK, SLANG, VERB }
NPN: { $, ADJ, AUX, COND, COP, NEG, NEG_PRE, PART, PREP, QMARK, SLANG, VERB }
NBAR: { $, ADJ, AUX, COND, COP, NEG, NEG_PRE, PART, PREP, QMARK, SLANG, VERB }
TAIL: { $ }
TAILW: { $, QMARK }
QM: { $ }
UTT': { $, PART, QMARK, SLANG }
ADJP': { $, COND, PART, QMARK, SLANG }
NBAR': { $, ADJ, AUX, COND, COP, NEG, NEG_PRE, PART, PREP, QMARK, SLANG, VERB }
CLAUSE': { $, COND, PART, QMARK, SLANG }
COMP': { $, COND, PART, QMARK, SLANG }
NBAR'': { $, ADJ, AUX, COND, COP, NEG, NEG_PRE, PART, PREP, QMARK, SLANG, VERB }

```

Figure 7: FOLLOW sets as printed by the program.

Comment

Worked example: FIRST(VP1) contains ϵ because VP1 may be empty (an intransitive verb such as chop in U don chop). So the parser needs FOLLOW(VP1), which contains CONJ, PART, QMARK, SLANG and \$, to know when to choose the empty rule.

Presentation clue: FIRST answers “which rule starts with this token?”; FOLLOW answers “when may I stop?”.

5.5 Step 4: the LL(1) parsing table

The table $M[X, a]$ tells the parser which rule to use when the symbol on top of the stack is X and the next token type is a . It has **153 filled cells**. A grammar is LL(1) only if no cell holds two different rules; the builder raises an error if that happens, and **none occurred**. Columns are split over two tables (part 1: ADJ, ADV, AUX, CONJ, COP, DET, NEG, NEG_PRE, NOUN; part 2: NUM, PART, PREP, PRON, QMARK, SLANG, VERB, \$). An empty cell means a syntax error.

Part 1

	ADJ	ADV	AUX	CONJ	COP	DET	NEG	NEG_PRE	NOUN
S	LEAD UTT TAIL	LEAD UTT TAIL		LEAD UTT TAIL	LEAD UTT TAIL	LEAD UTT TAIL		LEAD UTT TAIL	LEAD UTT TAIL
LEAD	ε	ε		ε	ε	ε		ε	ε
UTT	CLAUSE UTT'	CLAUSE UTT'		CONJ CLAUSE UTT'	CLAUSE UTT'	CLAUSE UTT'		CLAUSE UTT'	CLAUSE UTT'
CLAUSE	NP CLAUSE'	ADV ADJP			COP COMP	NP CLAUSE'		VP	NP CLAUSE'
PRED	ADJP		AUX AUXC		COP COMP		NEG NEGC	VP	
AUXC	ADJP							VP	
NEGC	ADJP							VP	
COMP	ADJP	ADV COMP'							
ADJP	ADJ ADJP'								
VP								NEG_PRE VERB NEG VP1	
VP1	NPN VP2	ADV VP2		ε		NPN VP2			NPN VP2
VPOBJ	NP VP2			VP2		NP VP2			NP VP2
VP2				ε					
PP									
NP	NPN					NPN			NPN
NPN	NBAR					DET NBAR			NBAR
NBAR	ADJ NBAR''								NOUN NBAR'
TAIL									
TAILW									
QM									
UTT'				CONJ CLAUSE UTT'					
ADJP'	ADJ ADJP'			ε					
NBAR'	ε		ε	ε	ε		ε	ε	NOUN NBAR'
CLAUSE'	PRED		PRED	ε	PRED		PRED	PRED	
COMP'	ADJP			ε					
NBAR''									NOUN NBAR'

Part 2

	NUM	PART	PREP	PRON	QMARK	SLANG	VERB	\$
S	LEAD UTT TAIL			LEAD UTT TAIL		LEAD UTT TAIL	LEAD UTT TAIL	
LEAD	ε			ε		SLANG	ε	
UTT	CLAUSE UTT'			CLAUSE UTT'			CLAUSE UTT'	
CLAUSE	NP CLAUSE'			NP CLAUSE'			VP	
PRED							VP	
AUXC							VP	
NEGC							VP	
COMP								
ADJP								
VP							VERB VP1	
VP1	NPN VP2	ε	PP VP2	PRON VPOBJ	ε	ε		ε
VPOBJ	NP VP2	VP2	VP2	NP VP2	VP2	VP2		VP2
VP2		ε	PP VP2		ε	ε		ε
PP			PREP NP					
NP	NPN			PRON				
NPN	NBAR							
NBAR	NUM NBAR'							
TAIL		TAILW QM			TAILW QM	TAILW QM		TAILW QM
TAILW		PART			ε	SLANG		ε
QM					QMARK			ε
UTT'		ε			ε	ε		ε
ADJP'		ε			ε	ε		ε
NBAR'		ε	ε		ε	ε	ε	ε
CLAUSE'		ε			ε	ε	PRED	ε
COMP'		ε			ε	ε		ε
NBAR''	NUM NBAR'							

Comment: how to read the table

Example 1: the row CLAUSE under column DET holds NP CLAUSE', so a clause starting with an article (le, my, that) is parsed as a noun phrase first.

Example 2: the row VP1 under column PREP holds PP VP2, so a verb followed directly by a preposition (Levam **for** this price) reads a prepositional phrase.

Example 3: the row VP2 under column CONJ holds ε, so after the verb phrase the parser stops and lets the conjunction make start a new clause.

6. Implementation

6.1 Architecture

File	Responsibility
vocabulary.csv	The lexicon: word, token type, language, concept and note. Edited by hand; drives the lexer.
lexer.py	Builds one regex rule per token type and scans a phrase into tokens (Section 4.2).
grammar.py	The raw CFG plus generic algorithms: left-recursion removal, left factoring, FIRST, FOLLOW and the LL(1) table.
parser.py	Table-driven LL(1) parser with a step trace, a parse tree and precise error messages.
corpus.csv / corpus.py	The 50 test utterances with translation, topic and expected verdict.
analysis.py	Token frequency, language mix, per-topic figures and concept variation (Section 4.4).
main.py	Command-line interface: parses the corpus and prints the grammar steps and statistics.
gui.py	Desktop interface (Tkinter) with tokens, parse tree, parse trace, results and grammar windows.
test_pipeline.py	Nine automated tests (Section 7.1).

6.2 The table-driven LL(1) parsing algorithm

The parser keeps a stack that starts with \$ S. It repeats the following until it accepts or fails:

```

X = top of stack ; a = kind of the next token
if X == $ and a == $ : ACCEPT
elif X is a terminal : if X == a then pop X and advance else ERROR (expected X)
else : rule = M[X, a]
      if rule is empty then ERROR (list the tokens M allows for X)
      else pop X and push the right-hand side of rule in reverse order

```

While it runs, the parser also builds the **parse tree** and records every step. On an error it reports the offending token and the token types that would have been accepted, for example: Syntax Error near token 'la' (unexpected DET; expected one of: \$, ADJ, AUX, ...).

6.3 Worked example: F04 “U don chop?”

The utterance means “Have you eaten?”. Tokens: U (PRON), don (AUX), chop (VERB), ? (QMARK). The trace below is generated by the program:

Step	Stack (top on the right)	Remaining input	Action
1	\$ S	PRON AUX VERB QMARK \$	S -> LEAD UTT TAIL
2	\$ TAIL UTT LEAD	PRON AUX VERB QMARK \$	LEAD -> ϵ
3	\$ TAIL UTT	PRON AUX VERB QMARK \$	UTT -> CLAUSE UTT'
4	\$ TAIL UTT' CLAUSE	PRON AUX VERB QMARK \$	CLAUSE -> NP CLAUSE'
5	\$ TAIL UTT' CLAUSE' NP	PRON AUX VERB QMARK \$	NP -> PRON
6	\$ TAIL UTT' CLAUSE' PRON	PRON AUX VERB QMARK \$	match PRON
7	\$ TAIL UTT' CLAUSE'	AUX VERB QMARK \$	CLAUSE' -> PRED
8	\$ TAIL UTT' PRED	AUX VERB QMARK \$	PRED -> AUX AUXC
9	\$ TAIL UTT' AUXC AUX	AUX VERB QMARK \$	match AUX
10	\$ TAIL UTT' AUXC	VERB QMARK \$	AUXC -> VP
11	\$ TAIL UTT' VP	VERB QMARK \$	VP -> VERB VP1
12	\$ TAIL UTT' VP1 VERB	VERB QMARK \$	match VERB
13	\$ TAIL UTT' VP1	QMARK \$	VP1 -> ϵ
14	\$ TAIL UTT'	QMARK \$	UTT' -> ϵ
15	\$ TAIL	QMARK \$	TAIL -> TAILW QM
16	\$ QM TAILW	QMARK \$	TAILW -> ϵ
17	\$ QM	QMARK \$	QM -> QMARK
18	\$ QMARK	QMARK \$	match QMARK

Step	Stack (top on the right)	Remaining input	Action
19	\$	\$	accept

YAOUNDE / LANGUAGE TOOLS

Urban Language Analyzer

PHRASE

U don chop?

Analyze phrase Clear Ctrl+Enter to analyze

50 COLLECTED UTTERANCES < Prev Next >

F04 U don chop?

Accepted

Syntax Valid (Accepted by CFG)

F04 · food · P07 · matches the expected ACCEPT
Note: Pidgin perfect marker 'don' + question mark.

Contributor's translation Have you eaten?

Tokens Parse tree **Parse trace**

Each row is one step of the LL(1) algorithm. The top of the stack is on the right.

Step	Stack	Remaining input	Action
1	\$ S	PRON AUX VERB QMARK \$	S -> LEAD UTT TAIL
2	\$ TAIL UTT LEAD	PRON AUX VERB QMARK \$	LEAD -> ε
3	\$ TAIL UTT	PRON AUX VERB QMARK \$	UTT -> CLAUSE UTT'
4	\$ TAIL UTT' CLAUSE	PRON AUX VERB QMARK \$	CLAUSE -> NP CLAUSE'
5	\$ TAIL UTT' CLAUSE' NP	PRON AUX VERB QMARK \$	NP -> PRON
6	\$ TAIL UTT' CLAUSE' PRON	PRON AUX VERB QMARK \$	match PRON
7	\$ TAIL UTT' CLAUSE'	AUX VERB QMARK \$	CLAUSE' -> PRED
8	\$ TAIL UTT' PRED	AUX VERB QMARK \$	PRED -> AUX AUXC
9	\$ TAIL UTT' AUXC AUX	AUX VERB QMARK \$	match AUX
10	\$ TAIL UTT' AUXC	VERB QMARK \$	AUXC -> VP
11	\$ TAIL UTT' VP	VERB QMARK \$	VP -> VERB VP1
12	\$ TAIL UTT' VP1 VERB	VERB QMARK \$	match VERB
13	\$ TAIL UTT' VP1	QMARK \$	VP1 -> ε
14	\$ TAIL UTT'	QMARK \$	UTT' -> ε
15	\$ TAIL	QMARK \$	TAIL -> TAILW QM
16	\$ QM TAILW	QMARK \$	TAILW -> ε
17	\$ QM	QMARK \$	QM -> QMARK

Figure 8: The same trace in the desktop interface (Parse trace tab).

YAOUNDE / LANGUAGE TOOLS

Urban Language Analyzer

PHRASE

U don chop?

Analyze phrase Clear Ctrl+Enter to analyze

50 COLLECTED UTTERANCES < Prev Next >

F04 U don chop?

Accepted

Syntax Valid (Accepted by CFG)

F04 · food · P07 · matches the expected ACCEPT
Note: Pidgin perfect marker 'don' + question mark.

Contributor's translation Have you eaten?

Tokens **Parse tree** Parse trace

Hide helper nodes (names ending in ') and empty nodes

```

graph TD
    S --> UTT
    UTT --> CLAUSE
    CLAUSE --> NP
    NP --> PRON["PRON 'U' (Pidgin)"]
    CLAUSE --> PRED
    PRED --> AUX["AUX 'don' (Pidgin)"]
    PRED --> AUXC
    AUXC --> VP
    VP --> VERB["VERB 'chop' (Pidgin)"]
    CLAUSE --> TAIL
    TAIL --> QM
    QM --> QMARK["QMARK '?' (Unknown)"]
  
```

Figure 9: The resulting parse tree (Parse tree tab), with helper nodes hidden.

Comment

Follow the steps: S expands to LEAD UTT TAIL; LEAD becomes ϵ ; UTT reads the clause; the subject U is matched, then don is recognised as an aspect auxiliary (PRED $\rightarrow$ AUX AUXC) and chop as the verb; finally TAIL matches the question mark and \$ ends the parse.

Presentation clue: the parse tree shows the structure the trace built. U is the subject NP, don chop the predicate and ? the tail.

7. Testing and results

7.1 Automated tests

Test	What it checks
test_dotted_abbreviation_is_one_token	E.N.E.O is scanned as one NOUN token, not four.
test_number_with_suffix	2k and 100k are NUM tokens.
test_two_word_exclamation	Ndon ya (extra spaces) is a single SLANG token.
test_question_mark_unknown_word_and_emoji	? is QMARK, an unknown word is WORD, an emoji is INVALID.
test_word_boundaries	sa1a is not split into a smaller word.
test_left_recursion_elimination	A $\rightarrow$ A a b becomes A $\rightarrow$ b A' and A' $\rightarrow$ a A' ϵ .
test_left_factoring	S $\rightarrow$ a b a c becomes S $\rightarrow$ a S' and S' $\rightarrow$ b c.
test_project_grammar_is_ll1_and_was_transformed	Both transformations changed our grammar and the LL(1) table was built without conflict.
test_every_collected_utterance_gets_its_expected_verdict	All 50 utterances get the verdict recorded in the corpus.

```

PS C:\Users\Samsimbom\compiler_construction_yaounde> py -m unittest -v
test_every_collected_utterance_gets_its_expected_verdict (test_pipeline.CorpusTests.test_every_collected_utterance_gets_its_expected_verdict) ...
ok
test_left_factoring (test_pipeline.GrammarAlgorithmTests.test_left_factoring) ... ok
test_left_recursion_elimination (test_pipeline.GrammarAlgorithmTests.test_left_recursion_elimination) ... ok
test_project_grammar_is_ll1_and_was_transformed (test_pipeline.GrammarAlgorithmTests.test_project_grammar_is_ll1_and_was_transformed) ... ok
test_dotted_abbreviation_is_one_token (test_pipeline.LexerTests.test_dotted_abbreviation_is_one_token) ... ok
test_number_with_suffix (test_pipeline.LexerTests.test_number_with_suffix) ... ok
test_question_mark_unknown_word_and_emoji (test_pipeline.LexerTests.test_question_mark_unknown_word_and_emoji) ... ok
test_two_word_exclamation (test_pipeline.LexerTests.test_two_word_exclamation) ... ok
test_word_boundaries (test_pipeline.LexerTests.test_word_boundaries) ... ok

-----
Ran 9 tests in 0.009s

OK

```

Figure 10: `py -m unittest -v`: all 9 tests pass.

Comment

The last test is the important one: it re-runs the whole pipeline on every collected utterance and fails if any verdict changes, so the results below cannot silently drift from what the grammar does.

7.2 Overall results

Topic	Utterances	Accepted	Rejected	Accepted share
Taxi / bendskin	6	4	2	66.7%
Money matters	8	4	4	50.0%
Rainy season	7	2	5	28.6%
Internet / network	6	4	2	66.7%
Market bargaining	6	4	2	66.7%
Electricity	4	4	0	100.0%
Phone credit & Mobile Money	3	3	0	100.0%
Chop / food	5	2	3	40.0%
Fuel queues	4	4	0	100.0%
Open topic	1	0	1	0.0%
Total	50	31	19	62.0%

```

Contributor's translation: Fuel is expensive

U03 [fuel, P07]: 'Petrol don short'
Tokens: [('Petrol', 'NOUN', 'EN'), ('don', 'AUX', 'PDG'), ('short', 'ADJ', 'EN')]
Result: Syntax Valid (Accepted by CFG)
Contributor's translation: The available amount of fuel can not go round

U04 [fuel, P09]: 'Petrol don deer'
Tokens: [('Petrol', 'NOUN', 'EN'), ('don', 'AUX', 'PDG'), ('deer', 'ADJ', 'PDG')]
Result: Syntax Valid (Accepted by CFG)
Contributor's translation: Petrol is now so expensive

O01 [open, P05]: 'Le pater la'
Tokens: [('Le', 'DET', 'FR'), ('pater', 'NOUN', 'FR'), ('la', 'DET', 'FR')]
Result: Syntax Error near token 'la' (unexpected DET; expected one of: $, ADJ, AUX, CONJ, COP, NEG, NEG_PRE, NOUN, PART, PREP, QMARK, SLANG, VERB) [Rejected]
Contributor's translation: The president
Note: Same homograph as T03: postposed 'la' (là) is read as the article.

Accepted 31, rejected 19 (of 50 parsed; 10 unusable form entries excluded); 0 differ from the expected label.

```

Figure 11: `py main.py --no-interactive`: the last entries and the summary line (31 accepted, 19 rejected, 0 differ from the expected label).

31 accepted, 19 rejected of 50 collected utterances. Double-click a row to open it.

ID	Topic	Utterance	Result	Why / note
T01	taxi	Fou le camp	Accepted	
T02	taxi	Yu no serios	Accepted	
T03	taxi	Enlève ta brouette la sur la route	Rejected	Postposed 'la' (là, 'there') is spelled like the article 'la', so the single-tag lexicon reads it as DET.
T04	taxi	Lef me for that achombo house	Accepted	
T05	taxi	I get na 400	Rejected	'na' works like a preposition/focus marker here: the grammar only models its copula use ('Na how').
T06	taxi	Na how	Accepted	
E01	money	Ne joue pas avec Mon argent salo	Accepted	French discontinuous negation ne...pas.
E02	money	That do small	Rejected	'that' is a demonstrative used as a pronoun; the lexicon tags it DET (as in 'that achombo house') and a bare DET is not a noun phrase.
E03	money	Gère moi non nkap	Rejected	'non' (probably a typo for 'mon', kept as typed) is read as NEG, which cannot follow the object.
E04	money	You get some twenty pongos for Dey ?	Rejected	'dey' is tagged VERB (as in 'Light no dey'; here it means a locative 'there', so the PP 'for Dey' fails.
E05	money	I send my aza make you fund me fapnkolo	Accepted	Two clauses joined by 'make'; 'fund' takes a pronoun then a noun (double object).
E06	money	A fit see some small 100k 🤔🤔🤔	Rejected	The clause itself is grammatical; the trailing emoji are invalid characters, which the lexer keeps and the parser rejects.
E07	money	I drop aza?	Accepted	First line of a two-line form cell.
E08	money	Fund my account abeg	Accepted	Second line of a two-line form cell.
R01	rain	Na which bad luck this too nor?	Rejected	A wh-cleft with a noun-phrase complement after 'na' is not modeled: the copula only takes an adjective or adverb complement.
R02	rain	Rain do worry bad	Rejected	Postverbal intensifier 'bad' is not modeled; after the verb it is read as the start of an object noun phrase.
R03	rain	Massah yacob il fait froid	Rejected	'yacob' is not in the vocabulary (unknown word).
R04	rain	Enter motomoto	Accepted	
R05	rain	Motomoto	Accepted	Single noun fragment.
R06	rain	Do rain so eh	Rejected	'do' is used as a main verb here (weather 'do rain'), but the lexicon tags it as an aspect auxiliary.
R07	rain	This rain so na weather four two and. A half	Rejected	The postposed 'so' after the subject is not modeled: the utterance also has a mid-sentence full stop and unclear wording.

Figure 12: The “All results” window: accepted rows are green, rejected rows red, each with its reason.

Comment

31 of 50 utterances (62.0%) are accepted. This figure is **optimistic**: the grammar was designed from these same utterances, so it measures how well the rules describe the sample, not how well they would generalise. The rejections are as informative as the acceptances (Section 7.4).

7.3 Accepted utterances

ID	Utterance	Contributor's translation	Structure recognised by the grammar
T01	Fou le camp	Leave this place	imperative verb phrase
T02	Yu no serios	You're not serious	subject + negation
T04	Lef me for that achombo house	Drop me off and that local restaurant	imperative verb phrase prepositional phrase
T06	Na how	How are you	subjectless copular clause
E01	Ne joue pas avec Mon argent salo	Don't play with my money fool	imperative verb phrase French ne...pas negation; prepositional phrase; final exclamation
E05	I send my aza make you fund me fapnkolo	Should I send my account details so you can send me 5000frs	subject + verb phrase subject + verb phrase clauses joined by a conjunction; double object
E07	I drop aza?	Should I send my account details?	subject + verb phrase question mark
E08	Fund my account abeg	Please can you send me some money	imperative verb phrase final exclamation
R04	Enter motomoto	Enter mud	imperative verb phrase
R05	Motomoto	Mud	noun-phrase fragment
I02	Network poor bad	The network is poor	subject + predicate adjective
I03	Orange et le réseau nkgian	Orange and weak internet	noun-phrase fragment subject + predicate adjective clauses joined by a conjunction

ID	Utterance	Contributor's translation	Structure recognised by the grammar
I05	Yehum MTN	Disappointed at MTN	noun-phrase fragment leading exclamation
I06	Make single pole go offline make network cam back	Let single people go offline so we can have good network 🌈	subject + verb phrase subject + verb phrase leading conjunction; clauses joined by a conjunction
K02	Levam for this price	Leave it at this price	imperative verb phrase prepositional phrase
K04	Last market	What they sell around the closing time of the market	noun-phrase fragment
K05	Last price	Minimum price the seller can't go Below	noun-phrase fragment
K06	Ei over deer	It's too expensive	adverb + adjective leading exclamation
L01	Massa Eneo	Ugh, this Eneo (a sigh of annoyance when Eneo cuts the electricity)	noun-phrase fragment leading exclamation
L02	E.N.E.O	Eneo has done it again	noun-phrase fragment
L03	Ndon ya SOCADEL	Disappointed at SOCADEL	noun-phrase fragment leading exclamation
L04	Light no dey	There's no light	subject + negation
C01	Gère ton OM	Give me your orange money	imperative verb phrase
C02	Abeg momo me	Please send me money through mobile money	imperative verb phrase leading exclamation
C03	Abeg momo me 2k	Please send me 2k via momo	imperative verb phrase leading exclamation; double object
F01	La nourriture si dja	This food is nice	subject + copula + complement
F04	U don chop?	Have you eaten?	subject + aspect marker (AUX) question mark
U01	Fair vite nor	Do fast	imperative verb phrase final particle
U02	Le carburant est chère	Fuel is expensive	subject + copula + complement
U03	Petrol don short	The available amount of fuel can not go round	subject + aspect marker (AUX)
U04	Petrol don deer	Petrol is now so expensive	subject + aspect marker (AUX)

YAOUNDE / LANGUAGE TOOLS

Urban Language Analyzer

PHRASE

I send my aza make you fund me fapnkolo

Analyze phrase Clear Ctrl+Enter to analyze

50 COLLECTED UTTERANCES < Prev Next >

E05 I send my aza make you fund me fapnkolo

Accepted

Syntax Valid (Accepted by CFG)

E05 · money · P07 · matches the expected ACCEPT
Note: Two clauses joined by 'make'; 'fund' takes a pronoun then a noun (double object).

Contributor's translation Should I send my account details so you can send me 5000frs

Tokens Parse tree Parse trace

#	Word	Token type	Language
1	I	PRON	English
2	send	VERB	English
3	my	DET	English
4	aza	NOUN	Slang
5	make	CONJ	Pidgin
6	you	PRON	English
7	fund	VERB	English
8	me	PRON	English
9	fapnkolo	NOUN	Slang

Figure 13: Accepted utterance E05 in the interface: tokens coloured by language (English blue, Pidgin yellow, slang green).

Comment

E05 (“I send my aza make you fund me fapnkolo”) shows the strengths of the grammar in one line: two clauses chained by the Pidgin conjunction make, a double object (fund me fapnkolo) and three languages (English, Pidgin, slang).

7.4 Rejected utterances and why

Each rejection is deliberate and documented. The column “Stops at” is the first token the parser could not accept.

ID	Utterance	Stops at	Reason
T03	Enlève ta brouette la sur la route	unexpected DET 'la'	Postposed 'la' (là, 'there') is spelled like the article 'la', so the single-tag lexicon reads it as DET.
T05	I get na 400	unexpected COP 'na'	'na' works like a preposition/focus marker here; the grammar only models its copula use ('Na how').
E02	That do small	unexpected AUX 'do'	'that' is a demonstrative used as a pronoun; the lexicon tags it DET (as in 'that achombo house') and a bare DET is not a noun phrase.
E03	Gère moi non nkap	unexpected NEG 'non'	'non' (probably a typo for 'mon', kept as typed) is read as NEG, which cannot follow the object.
E04	You get some twenty pongos for Dey ?	unexpected VERB 'Dey'	'dey' is tagged VERB (as in 'Light no dey'); here it means a locative 'there', so the PP 'for Dey' fails.
E06	A fit see some small 100k 🤔🤔🤔	invalid character '🤔'	The clause itself is grammatical; the trailing emoji are invalid characters, which the lexer keeps and the parser rejects.
R01	Na which bad luck this too nor?	unexpected DET 'which'	A wh-cleft with a noun-phrase complement after 'na' is not modeled: the copula only takes an adjective or adverb complement.
R02	Rain do worry bad	end of input	Postverbal intensifier 'bad' is not modeled; after the verb it is read as the start of an object noun phrase.
R03	Massah yacob il fait froid	unknown word 'yacob'	'yacob' is not in the vocabulary (unknown word).
R06	Do rain so eh	unexpected AUX 'Do'	'do' is used as a main verb here (weather 'do rain'), but the lexicon tags it as an aspect auxiliary.
R07	This rain so na weather four two and. A half	unexpected ADV 'so'	The postposed 'so' after the subject is not modeled; the utterance also has a mid-sentence full stop and unclear wording.
I01	Mtn Va tous nous montrer au pays ci	unexpected DET 'tous'	The floating quantifier 'tous' between the auxiliary 'va' and the verb is not modeled.
I04	MTN ns craze	unknown word 'ns'	'ns' is not in the vocabulary (unknown word).
K01	Venez me tromper	unexpected VERB 'tromper'	Serial verbs ('venez' + 'tromper') are not modeled: after a verb and pronoun object no second verb is allowed.
K03	Majumbum depuis longtemps	unexpected PREP 'depuis'	A verbless noun phrase followed by a prepositional phrase is not modeled.
F02	Chop rice chop rice	unexpected VERB 'chop'	Two juxtaposed imperatives with no conjunction are not modeled.
F03	I di find. Grind	invalid character '.'	The mid-utterance full stop is an invalid character for the grammar.
F05	You get chop for house	unexpected VERB 'chop'	'chop' is tagged VERB (its usual use in this data); here it is a noun ('food') right after another verb, which a single-tag lexicon cannot resolve.
O01	Le pater la	unexpected DET 'la'	Same homograph as T03: postposed 'la' (là) is read as the article.

YAOUNDE / LANGUAGE TOOLS

Urban Language Analyzer

PHRASE

Enlève ta brouette la sur la route

Analyze phrase Clear Ctrl+Enter to analyze

50 COLLECTED UTTERANCES Prev Next >

T03 Enlève ta brouette la sur la route

Rejected

Syntax Error near token 'la' (unexpected DET; expected one of: \$, ADJ, AUX, CONJ, COP, NEG, NEG_PRE, NOUN, PART, PREP, QMARK, SLANG, VERB) [Rejected]

T03 · taxi · P05 · matches the expected REJECT
Why: Postposed 'la' (là, 'there') is spelled like the article 'la', so the single-tag lexicon reads it as DET.

Contributor's translation Remove your trash car from the road

Tokens Parse tree Parse trace

#	Word	Token type	Language
1	Enlève	VERB	French
2	ta	DET	French
3	brouette	NOUN	French
4	la	DET	French
5	sur	PREP	French
6	la	DET	French
7	route	NOUN	French

Figure 14: Rejected T03: the red row is the first 'la', where the parser stopped; the panel shows the reason.

YAOUNDE / LANGUAGE TOOLS

Urban Language Analyzer

PHRASE

Massah yacob il fait froid

Analyze phrase Clear Ctrl+Enter to analyze

50 COLLECTED UTTERANCES Prev Next >

R03 Massah yacob il fait froid

Rejected

Syntax Error near token 'yacob' (unknown word, not in the vocabulary; expected one of: ADJ, ADV, CONJ, COP, DET, NEG_PRE, NOUN, NUM, PRON, VERB) [Rejected]

R03 · rain · P05 · matches the expected REJECT
Why: 'yacob' is not in the vocabulary (unknown word).

Contributor's translation Places are cold

Tokens Parse tree Parse trace

#	Word	Token type	Language
1	Massah	SLANG	Slang
2	yacob	WORD	Unknown
3	il	PRON	French
4	fait	VERB	French
5	froid	ADJ	French

Figure 15: Rejected R03: 'yacob' is not in the vocabulary, so it is tagged WORD (unknown).

Urban Language Analyzer

PHRASE

A fit see some small 100k👉👉👉

Analyze phrase Clear Ctrl+Enter to analyze

50 COLLECTED UTTERANCES

E06 A fit see some small 100k👉👉👉

Rejected

Syntax Error near token '👉' (invalid character: expected one of: \$, ADJ, AUX, CONJ, COP, NEG, NEG_PRE, NOUN, PART, PREP, QMARK, SLANG, VERB) [Rejected]

E06 · money · P09 · matches the expected REJECT
Why: The clause itself is grammatical; the trailing emoji are invalid characters, which the lexer keeps and the parser rejects.

Contributor's translation Could you please borrow me some money ,100k perhaps👉👉👉

#	Word	Token type	Language
1	A	PRON	Pidgin
2	fit	AUX	Pidgin
3	see	VERB	English
4	some	DET	English
5	small	ADJ	English
6	100k	NUM	Unknown
7	👉	INVALID	Unknown
8	👉	INVALID	Unknown
9	👉	INVALID	Unknown

Figure 16: Rejected E06: the clause is valid but the three emoji are INVALID characters.

Comment: three kinds of rejection

Out-of-vocabulary or invalid input (R03 yacob, I04 ns, E06 emoji, F03 full stop): a lexical problem, detected by the lexer.

Words with several roles (T03/O01 1a, F05 chop, E04 dey, T05 na, E02 that, R06 do): the lexicon gives each word one tag, so the grammar cannot tell which role is meant.

Constructions not modelled (K01 serial verbs, F02 juxtaposed imperatives, K03 verbless phrase + PP, R01 wh-cleft, R02 postverbal bad, I01 floating tous, E03 non): valid speech that lies outside our grammar fragment.

8. Why Yaoundé communication is linguistically complex

The results above are not only technical; they show how the speech itself resists a conventional compiler front end.

8.1 Several languages in one sentence

English and French are both official, and Pidgin, local languages and youth slang sit alongside them. In our data only 43.3% of tagged tokens are English and 37 of 50 utterances mix two or more languages. E01, “Ne joue pas avec Mon argent salo”, wraps a slang insult (salo) in a French negation frame; T04 puts a Pidgin verb, English function words and a slang noun (achombo) in one noun phrase.

8.2 One form, several functions

The same spelling plays different grammatical roles. 1a is an article in “la nourriture” and a postposed “there” in “Le pater la”; chop is a verb in “U don chop?” and a noun (food) in “You get chop for house”; dey is an existential verb in “Light no dey” and a locative “there” in “for Dey”; na is a copula in “Na how” and behaves like a preposition in “I get na 400”. A lexer that assigns one tag per word, followed by an LL(1) parser with one token of lookahead, cannot resolve these without context.

8.3 No standard spelling

Words are written as they sound: serios, deer for “dear” (expensive), nourriture, fair for “faire”, levam for “leave am”. The concept-variation table shows the same idea written many ways (money: argent, nkap, pongos, fapnkolo, majumbum). Every variant has to be listed or normalised.

8.4 Creative, short-lived vocabulary

Slang such as nkap, aza, pongos, fapnkolo, achombo, motomoto and ndon ya appears in no standard dictionary; meanings come from the contributors’ own translations. Names of companies and services (MTN, Orange, Eneo, SOCADEL, momo, OM) are part of daily speech too. Vocabulary changes quickly, so a fixed lexicon ages fast.

8.5 Oral, informal structure

The data are full of fragments (Motomoto; Last price), serial verbs (Venez me tromper), sentence-final particles (nor, eh), repetition (Chop rice chop rice), postposed particles (so, la) and expressive emoji. Few utterances are complete subject-verb-object sentences, so a grammar built for standard sentences covers only part of the speech.

8.6 What this means for a compiler design

A context-free grammar and a deterministic parser are good for the regular core and for teaching. Full coverage would need contextual tagging (to resolve la, chop, dey, na), spelling normalisation, and an ambiguity-tolerant parser (Earley or GLR) or a statistical model. Our rejected utterances point directly at where each of these would be needed.

9. Limitations and future work

- **Data:** 50 written utterances from one survey. The brief expects transcribed speech; ours are typed descriptions, and two topics (security checkpoints, university life) are missing.
- **Evaluation:** the grammar and lexicon were designed on the same 50 utterances, so the acceptance rate and the low out-of-vocabulary count are optimistic. A held-out test set is needed.
- **Single-tag lexicon:** each word has one token type, which causes the la/chop/dey/na rejections.
- **Scope of the grammar:** no serial verbs, wh-clefts, postposed particles or verbless phrase-plus-PP.
- **Future work:** collect and transcribe spoken data; add contextual tagging; try an SLR(1) or Earley parser and compare; add spelling normalisation.

10. Conclusion

We built a working analyzer for informal Yaoundé speech: a regular-expression lexer with a data-driven lexicon and language tags, a CFG that was transformed into an LL(1) grammar with FIRST/FOLLOW sets and a conflict-free parsing table, and a table-driven parser with a trace, a parse tree and precise errors. On 50 real utterances it accepted 31 and rejected 19, and every rejection is explained. The rejections show why Yaoundé speech is hard to analyse: heavy code-mixing, words with several roles, unstable spelling and fast-changing slang.

References

- [1] Aho, A. V., Lam, M. S., Sethi, R. and Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools, 2nd edition. Pearson.
- [2] Kießling, R. and Mous, M. (2004). Urban youth languages in Africa. *Anthropological Linguistics*, 46(3), 303-341.
- [3] Python Software Foundation. The re module documentation (regular expression operations), python.org.
- [4] The ICT University, Faculty of ICT (2026). CS4110 Compiler Construction final examination brief, Summer 2026.

Appendix A: Vocabulary

The 134 entries of `vocabulary.csv` (word, token type, language). Entries for si and dja were inferred from the contributors' word-by-word translations and should be confirmed.

Word	Type	Lang	Word	Type	Lang	Word	Type	Lang
bad	ADJ	EN	non	NEG	FR	avec	PREP	FR
chère	ADJ	FR	pas	NEG	FR	depuis	PREP	FR
craze	ADJ	PDG	ne	NEG_PRE	FR	for	PREP	EN
deer	ADJ	PDG	account	NOUN	EN	sur	PREP	FR
dja	ADJ	UNK	achombo	NOUN	SLG	a	PRON	PDG
froid	ADJ	FR	argent	NOUN	FR	i	PRON	EN

Word	Type	Lang	Word	Type	Lang	Word	Type	Lang
last	ADJ	EN	aza	NOUN	SLG	il	PRON	FR
nkangian	ADJ	SLG	brouette	NOUN	FR	me	PRON	EN
poor	ADJ	EN	camp	NOUN	FR	moi	PRON	FR
serios	ADJ	PDG	carburant	NOUN	FR	nous	PRON	FR
short	ADJ	EN	e.n.e.o	NOUN	EN	u	PRON	PDG
single	ADJ	EN	eneo	NOUN	EN	you	PRON	EN
small	ADJ	EN	fapnkolo	NOUN	SLG	yu	PRON	PDG
back	ADV	EN	grind	NOUN	PDG	abeg	SLANG	PDG
ci	ADV	FR	house	NOUN	EN	ei	SLANG	PDG
how	ADV	EN	light	NOUN	EN	massa	SLANG	SLG
longtemps	ADV	FR	luck	NOUN	EN	massah	SLANG	SLG
offline	ADV	EN	majumbum	NOUN	SLG	ndon ya	SLANG	SLG
over	ADV	EN	market	NOUN	EN	salo	SLANG	SLG
so	ADV	EN	motomoto	NOUN	SLG	yehum	SLANG	SLG
too	ADV	EN	mtn	NOUN	EN	cam	VERB	PDG
vite	ADV	FR	network	NOUN	EN	chop	VERB	PDG
di	AUX	PDG	nkap	NOUN	SLG	dey	VERB	PDG
do	AUX	PDG	nouriture	NOUN	FR	drop	VERB	EN
don	AUX	PDG	om	NOUN	FR	enlève	VERB	FR
fit	AUX	PDG	orange	NOUN	EN	enter	VERB	EN
va	AUX	FR	pater	NOUN	FR	fair	VERB	FR
and	CONJ	EN	pays	NOUN	FR	fait	VERB	FR
et	CONJ	FR	petrol	NOUN	EN	find	VERB	EN
make	CONJ	PDG	pole	NOUN	PDG	fou	VERB	FR
est	COP	FR	pongo	NOUN	SLG	fund	VERB	EN
na	COP	PDG	pongos	NOUN	SLG	gère	VERB	FR
si	COP	UNK	price	NOUN	EN	get	VERB	EN
la	DET	FR	rain	NOUN	EN	go	VERB	EN
le	DET	FR	réseau	NOUN	FR	joue	VERB	FR
mon	DET	FR	rice	NOUN	EN	lef	VERB	PDG
my	DET	EN	route	NOUN	FR	levam	VERB	PDG
some	DET	EN	socadel	NOUN	EN	momo	VERB	PDG
ta	DET	FR	weather	NOUN	EN	montrer	VERB	FR
that	DET	EN	four	NUM	EN	see	VERB	EN
this	DET	EN	twenty	NUM	EN	send	VERB	EN
ton	DET	FR	two	NUM	EN	tromper	VERB	FR
tous	DET	FR	eh	PART	PDG	venez	VERB	FR
which	DET	EN	nor	PART	PDG	worry	VERB	EN
no	NEG	PDG	au	PREP	FR			

Appendix B: Running the project

py gui.py	desktop interface
py main.py --no-interactive	parse all 50 utterances
py main.py --grammar	grammar steps, FIRST/FOLLOW and LL(1) table
py main.py --stats	token frequency and variation
py -m unittest -v	run the 9 automated tests

Python 3.9 or newer is required; the only optional component is Tkinter for the desktop interface.